

16 Bit CPU Project

April - 2025

Abstract

This report presents the design and implementation of a 16-bit RISC-like CPU. The project involves developing a pipelined processor with separate stages for instruction fetch, decode, execute, memory access, and write-back. The implemented CPU supports various instruction formats, including arithmetic, logical, memory, and branch operations. The report outlines the architecture, data hazard handling, control logic, and synthesis results. Simulation and synthesis results are provided to validate functionality and performance.

Table of Contents

Introduction.....	4
Design Specifications.....	4
Key Ideas.....	4
Objective.....	4
Body.....	4
Format A Design.....	5
Format B Design.....	5
Format L Design.....	6
Full Design.....	6
Fetch Stage Diagram:.....	7
Decode Stage Diagram:.....	8
Execute Stage Diagram:.....	9
Memory Stage Diagram:.....	10
Write Back Stage Diagram:.....	11
Results.....	11
Format A.....	11
Format B.....	12
Format L.....	14
Conclusion.....	16
Appendices.....	16
Full CPU Schematic.....	17

Introduction

Design Specifications

The objective of this project is to design and implement a 16-bit RISC CPU with a five-stage pipeline. The CPU supports arithmetic, logical, load/store, and branch instructions. The design includes hazard handling mechanisms and an efficient control unit to ensure correct execution.

Key Ideas

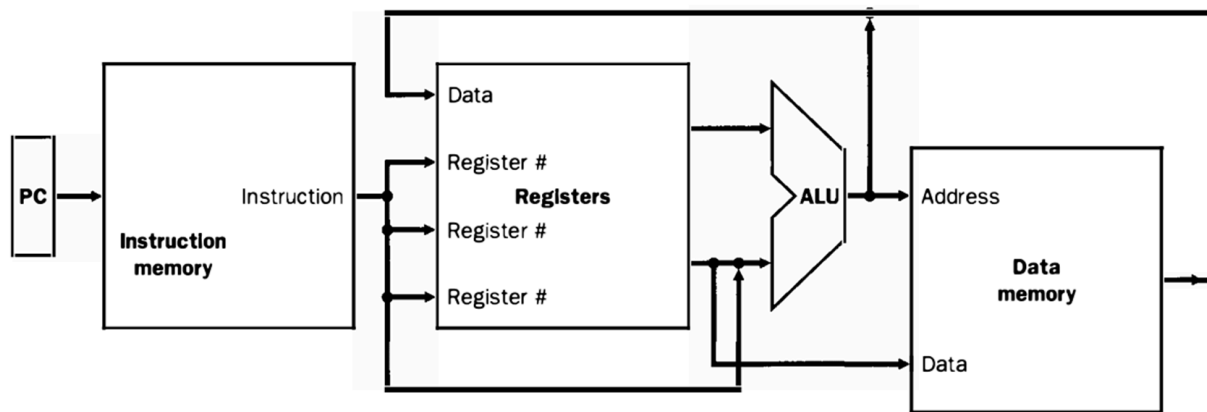
- **Pipelining:** The CPU follows a five-stage pipeline (Fetch, Decode, Execute, Memory, Write-back).
- **Instruction Set:** Supports essential arithmetic, branch, and memory operations.
- **Hazard Handling:** Implements stalling techniques.
- **VHDL Implementation:** Synthesizable design verified through simulation.

Objective

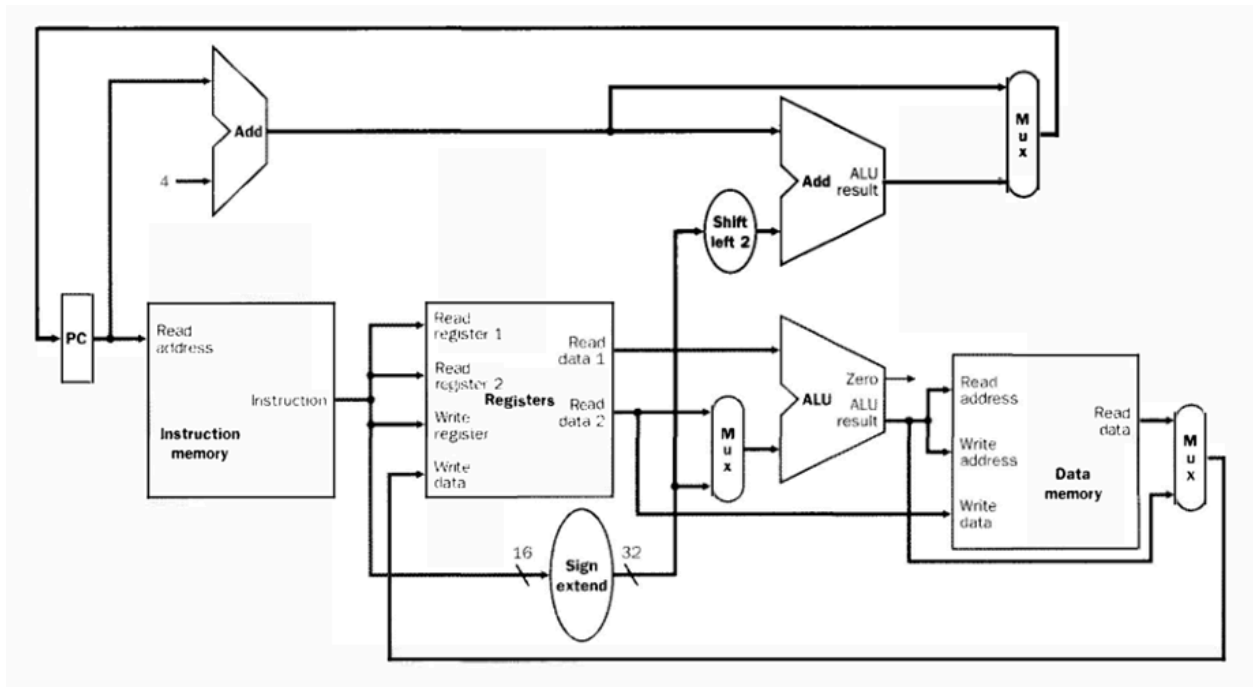
This project aims to design, implement, and test a 16-bit RISC CPU with pipelined execution. The CPU must support a defined instruction set while handling control and data hazards efficiently.

Body

Format A Design



Format B Design



Format L Design

```
-- Write index mux
wr_index_d <= instr_decoded_d.r_dest when (instr_decoded_d.opcode = LOAD or instr_decoded_d.opcode = MOV) else
instr_decoded_d.ra when decode_ctl_d.reg_dst = '0' else
"111";

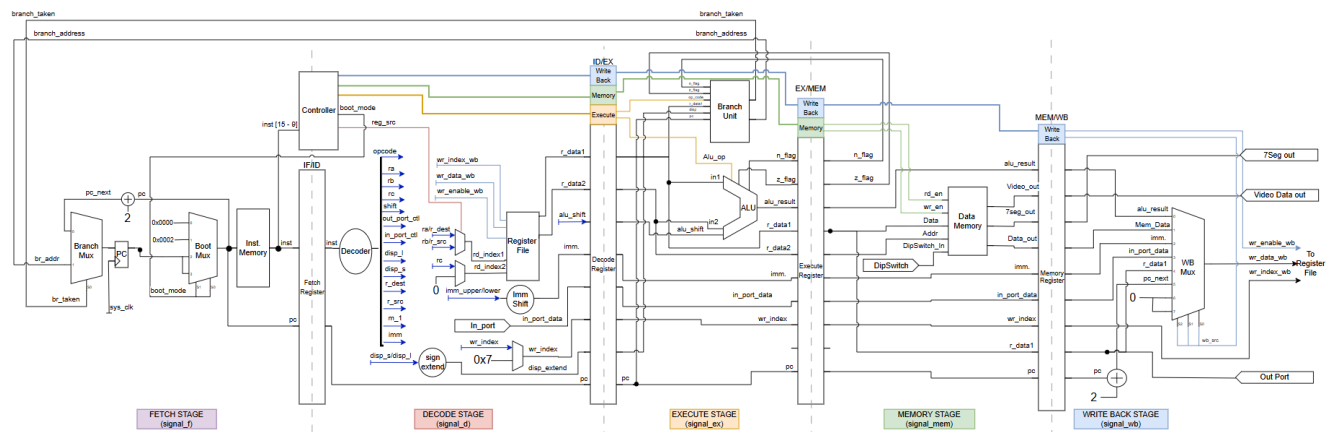
-- register index mux
rd_index1_d <= instr_decoded_d.r_src when (instr_decoded_d.opcode = LOAD or instr_decoded_d.opcode = MOV) else
instr_decoded_d.r_dest when (instr_decoded_d.opcode = STORE) else
instr_decoded_d.rb when decode_ctl_d.reg_src = '0' else
"111" when (decode_ctl_d.reg_src = '1') and (instr_decoded_d.opcode = RETURN_OP) else
instr_decoded_d.ra;

rd_index2_d <= instr_decoded_d.r_src when (instr_decoded_d.opcode = STORE) else
instr_decoded_d.rc when decode_ctl_d.reg_src = '0' else "000";

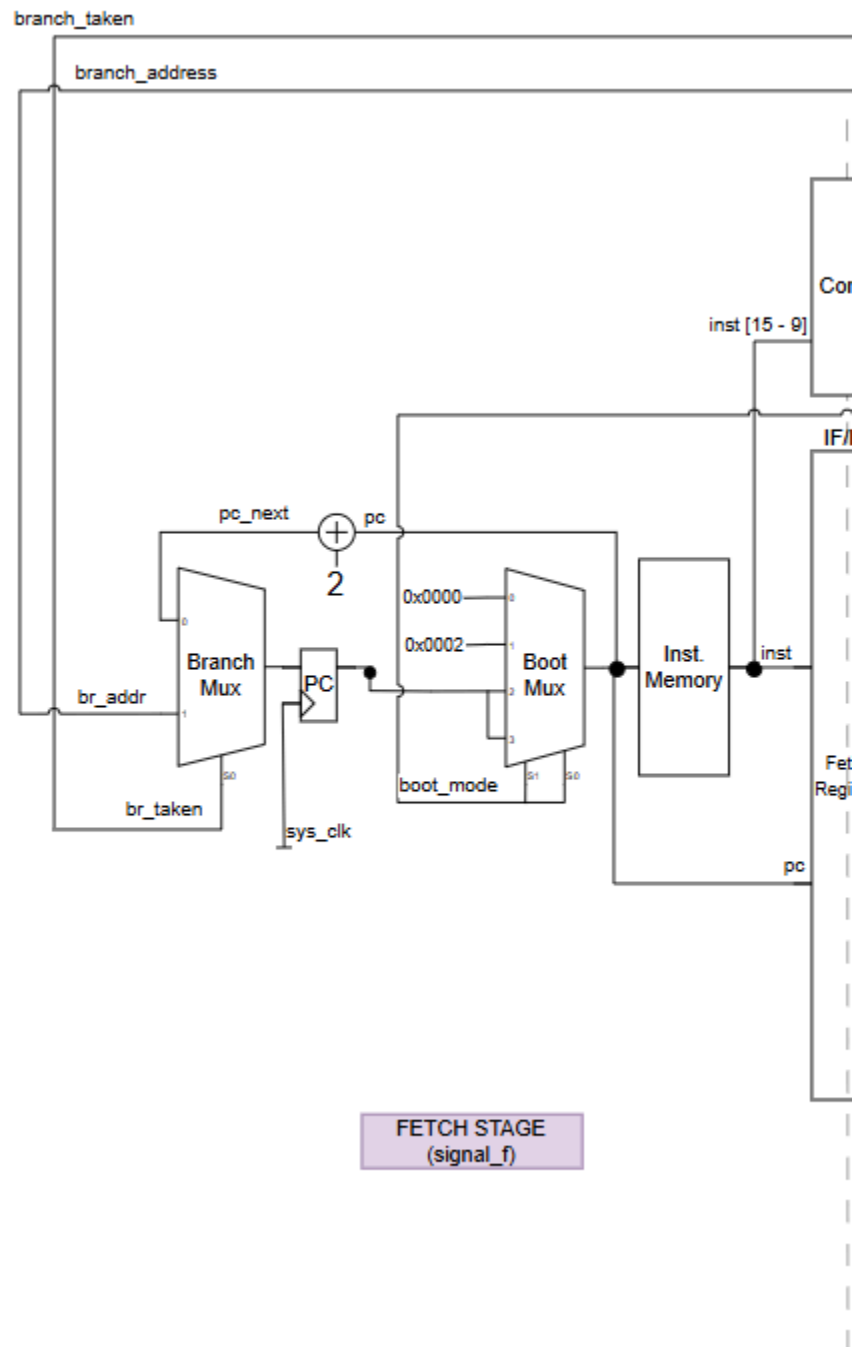
-- register file write data mux
wr_data_d <= memory_data_wb when write_back_ctl_wb.wb_src = MEMORY_DATA else
alu_result_wb when write_back_ctl_wb.wb_src = ALU_RES else
imm_cat_wb when write_back_ctl_wb.wb_src = IMM_FWD else
std_logic_vector(unsigned(pc_current_wb) + 2) when write_back_ctl_wb.wb_src = RETURN_PC else
inport_fwd_wb when write_back_ctl_wb.wb_src = INPORT_FWD else
rd_data1_wb when write_back_ctl_wb.wb_src = MOV_REG else
X"0000";
```

Full Design

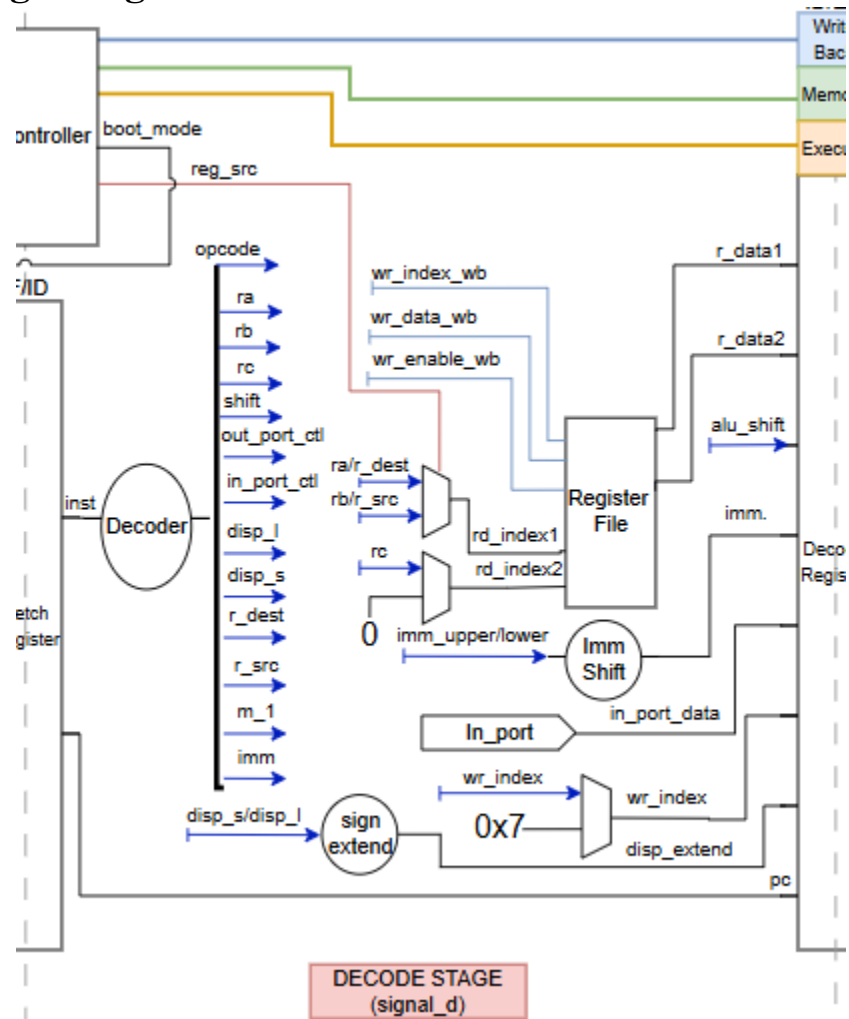
Not Pictured - Hazard unit



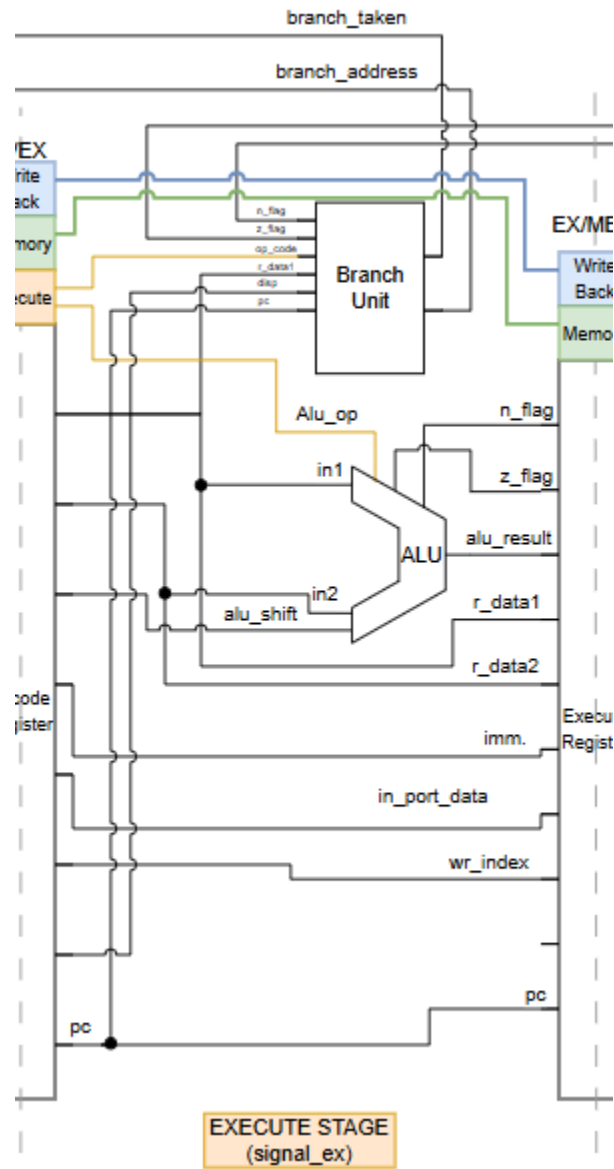
Fetch Stage Diagram:



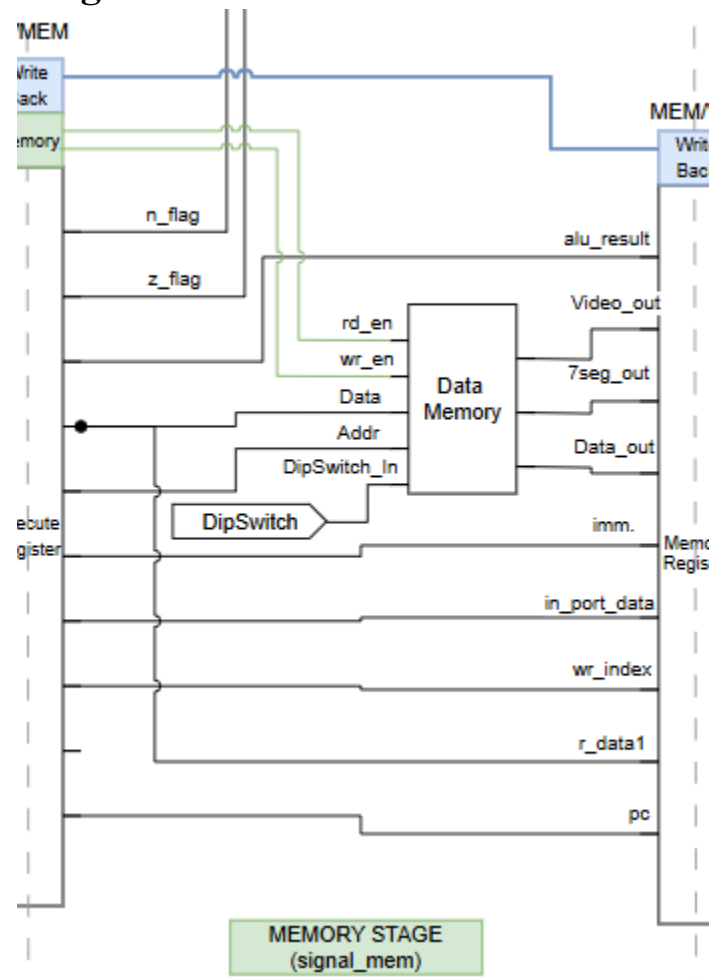
Decode Stage Diagram:



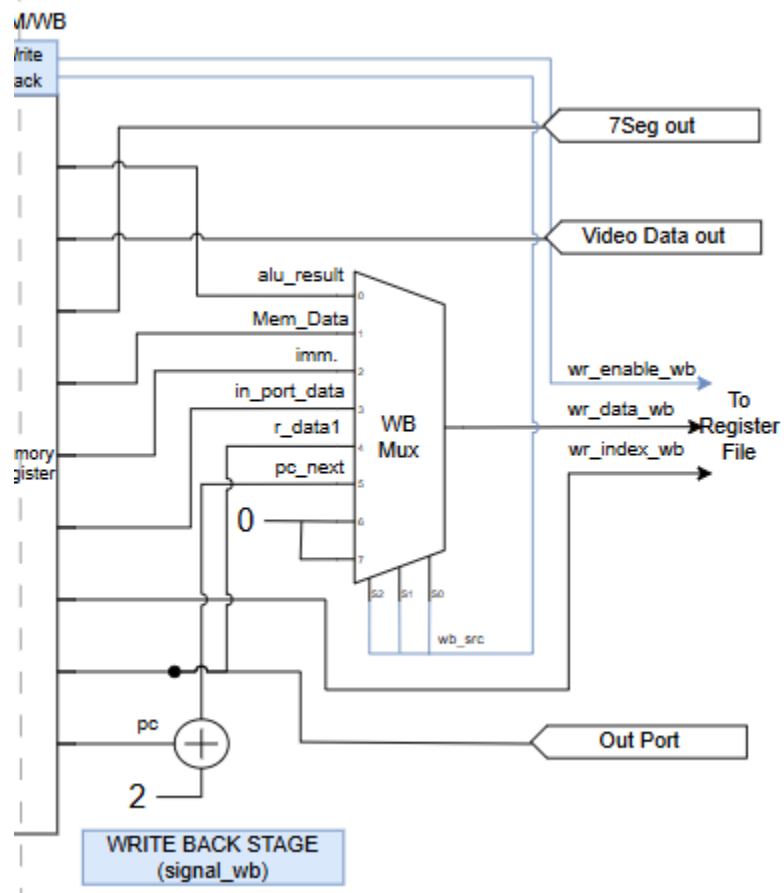
Execute Stage Diagram:



Memory Stage Diagram:



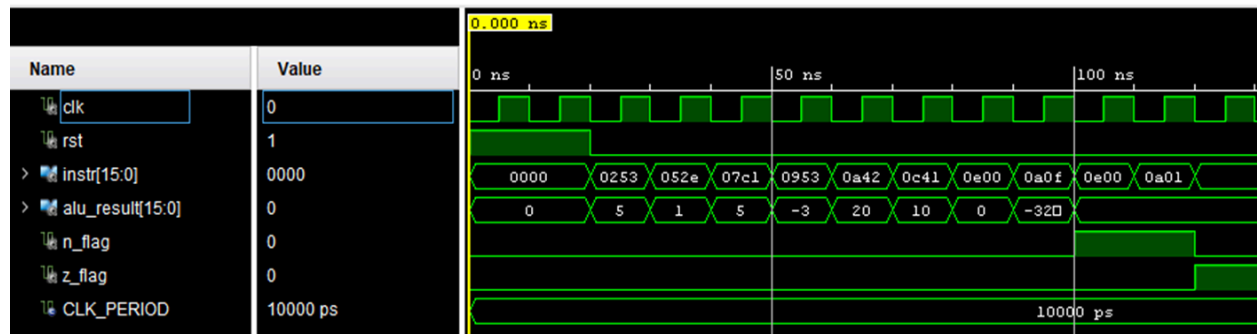
Write Back Stage Diagram:



Results

Format A

Test:1 - Format A					Binary			Decimal			Expected		Testbench		
Instruction #	Code	Function	OpCode	Format	ra	rb	rc	ra	rb	rc	Result	Flag (z/n)	Result	Flag	Pass?
0	ADD R1, R2, R3	R1 = R2 + R3	0000001	A1	001	010	011	1	2	3	5	x	5		y
1	SUB R4, R5, R6	R4 = R5 - R6	0000010	A1	100	101	110	4	5	6	1	x	1		y
2	MUL R7, R0, R1	R7 = R0 * R1	0000011	A1	111	000	001	7	0	1	5	x	5		y
3	NAND R5, R2, R3	R5 = R2 NAND R3	0000100	A1	101	010	011	5	2	3	-3	x	-3		y
4	SHL R1, #2	R1 = R1 << 2	0000101	A2	001	00	0010	1	0	2	20	x	20		y
5	SHR R1, #1	R1 = R1 >> 1	0000110	A2	001	00	0001	1	0	1	10	x	10		y
6	TEST R0	Check if R0 is zero or negative	0000111	A3	000	000	000	0	0	0	x	0	0	0	y
7	SHL R0, #15	R0 = R0 << 15	0000101	A2	000	00	1111	0	0	15	-32768	x	-32768	x	y
8	TEST R0	Check if R0 is zero or negative	0000111	A3	000	000	000	0	0	0	x	0	1	0	y
9	SHL R0, #1	R0 = R0 << 1	0000101	A2	000	00	0001	0	0	1	1	x	1	x	y
10	TEST R0	Check if R0 is zero or negative	0000111	A3	000	000	000	0	0	0	x	1	0	1	y



Register File				
R_n	init value	0	1	2
0	1	-32768	0	
1	0	5	20	10
2	2			
3	3			
4	0	1		
5	5	-3		
6	4			
7	0	5		

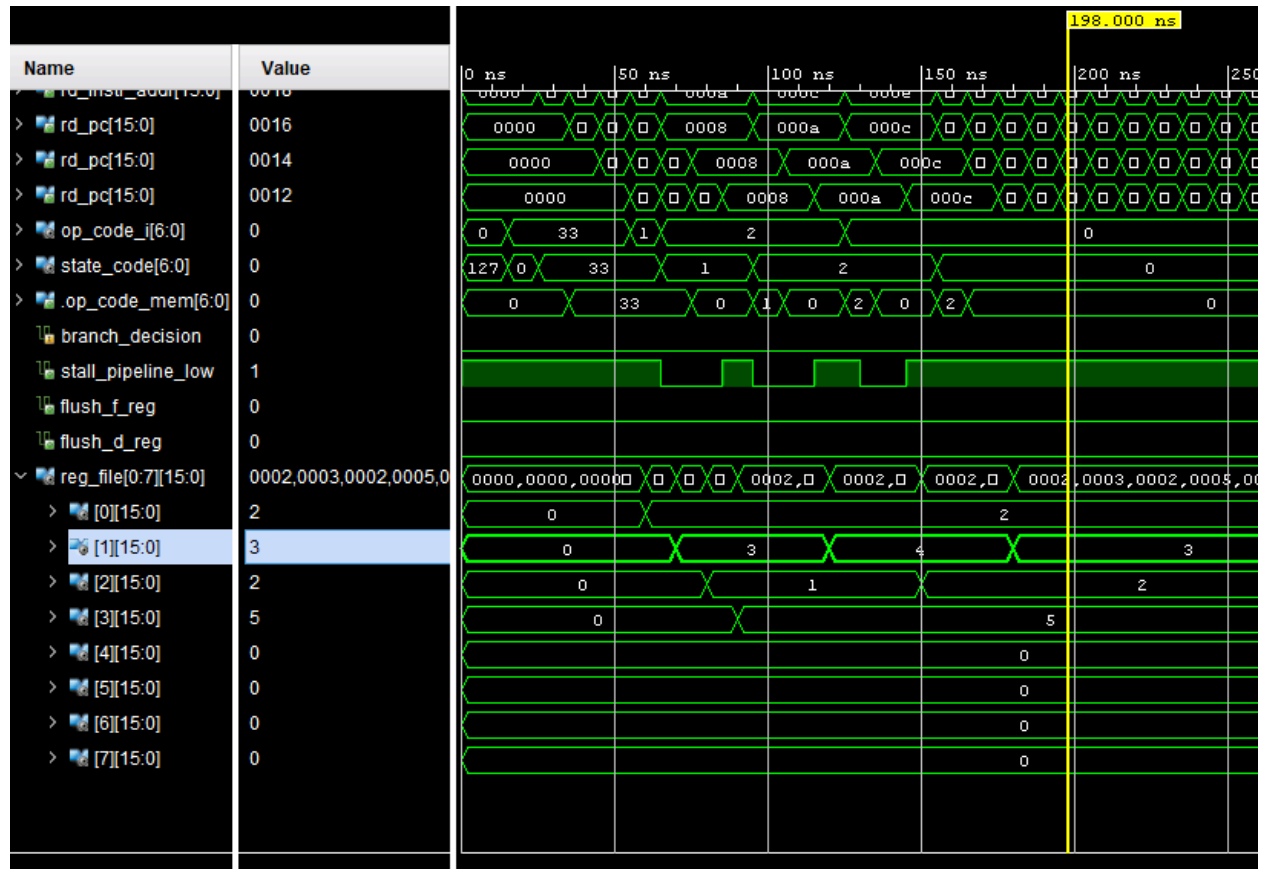
Format B

```

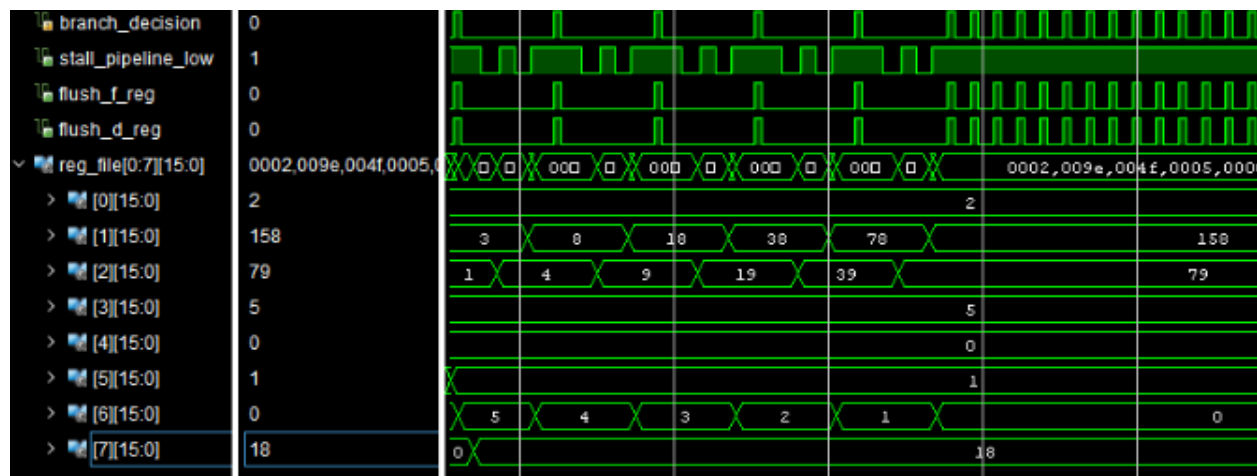
ORG 0x0210
.CODE
    IN R0 ; 02 ;
    IN R1 ; 03 ;
    IN R2 ; 01
    IN R3 ; 05 ;
    ADD R1, R1, R2
    SUB R2, R1, R0
    SUB R1, R3, R2

END

```

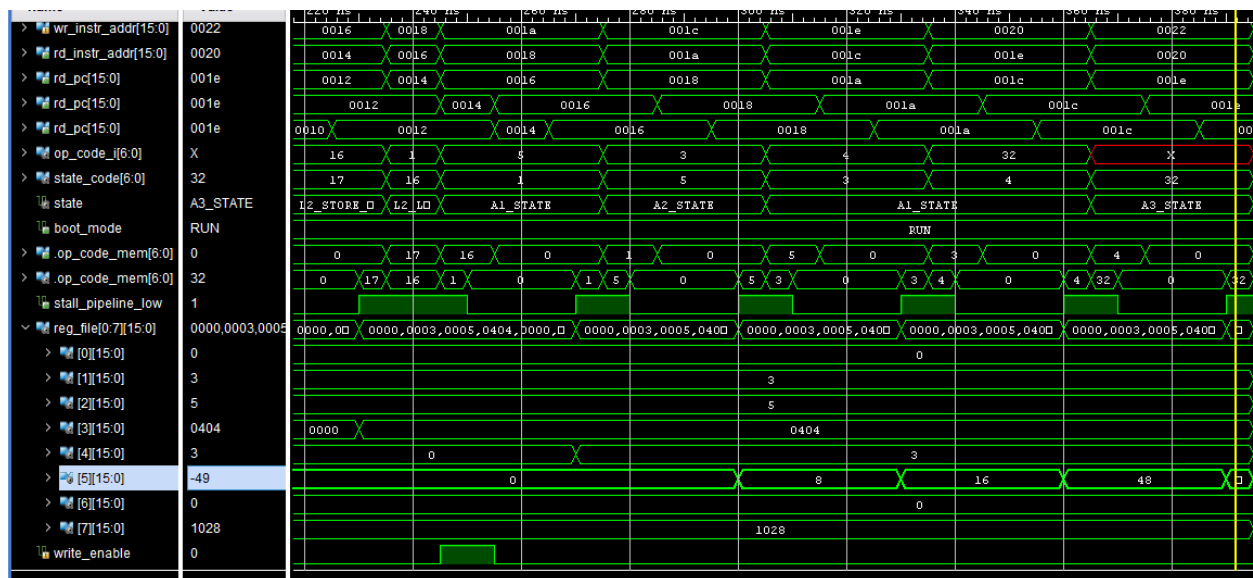


hex	
4200	IN R0; 02
4240	IN R1; 03
4280	IN R2; 01
42C0	IN R3; 05
4300	IN R4; 0
4340	IN R5; 01
4380	IN R6; 05
43C0	IN R7; 00
8D0A	BR.SUB R4,10
8000	BRR 0
028D	ADD R2, R1, R5
0642	MUL R1, R0, R2
05B5	SUB R6, R6, R5
0F80	TEST R6
8402	BRR.z 2
81FB	BRR -5
8E00	RETURN



Format L

hex	Asm
2500	LOADIMM.UPPER 0x00
2403	LOADIMM.LOWER 0x03
2678	MOV R1,R7
2500	LOADIMM.UPPER 0x00
2405	LOADIMM.LOWER 0x05
26B8	MOV R2,R7
2404	LOADIMM.LOWER 4
2504	LOADIMM.UPPER 4
26F8	MOV R3,R7
22C8	STORE R3,R1
2118	LOAD R4,R3
0354	ADD R5,R2,R4
0B41	SHL R5,1
0769	MUL R5,R5,R1
096D	NAND R5,R5,R5
4140	OUT R5

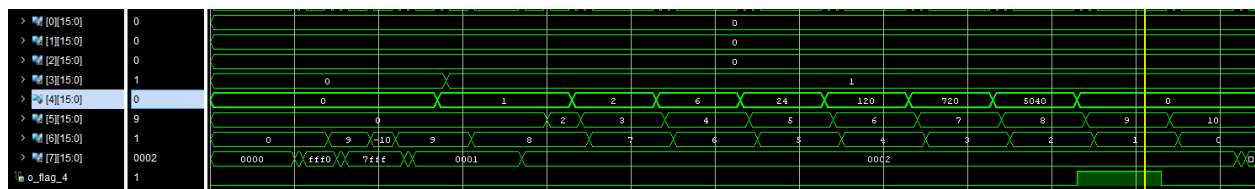


Final Test

```

0410 - 25FF main:      loadimm.upper DipSwitches.hi
0412 - 24F0           loadimm.lower DipSwitches.lo
0414 - 21B8           load      r6,r7
0416 - 257F           loadimm.upper DipSwitchMask.hi
0418 - 24FF           loadimm.lower DipSwitchMask.lo
041A - 09B7           nand      r6,r6,r7
041C - 09B6           nand      r6,r6,r6
041E - 2500           loadimm.upper 0x00
0420 - 2401           loadimm.lower 0x01
0422 - 2738           mov      r4,r7
0424 - 26F8           mov      r3,r7
0426 - 0F80           test     r6
0428 - 840D           brr.z   Done
042A - 05B3           sub      r6,r6,r3
042C - 0F80           test     r6
042E - 840A           brr.z   Done
0430 - 2500           loadimm.upper 0x00
0432 - 2402           loadimm.lower 0x02
0434 - 2778           mov      r5,r7
0436 - 0725 loop:     mul      r4,r4,r5
0438 - 036B           add      r5,r5,r3
043A - 05B3           sub      r6,r6,r3
043C - 0F80           test     r6
043E - 8402           brr.z   Done
0440 - 81FB           brr      loop
0442 - 25FF Done:     loadimm.upper LedDisplay.hi
0444 - 24F2           loadimm.lower LedDisplay.lo
0446 - 23E0           store   r7,r4
0448 - 81FD           brr      Done

```



Conclusion

In conclusion, we successfully implemented a 16-bit pipelined CPU. The design achieved functional correctness, demonstrating effective instruction execution and hazard management. Future improvements include optimizing pipeline efficiency and expanding the instruction set.

Appendices

Project repo: <https://github.com/Ayyymannn/16-Bit-CPU>

Full CPU Schematic

